



Développement d'une Interface Homme-Machine pour le Monitoring Infrastructure de Charge Électrique

Rapport de Stage

Stagiaire : Obeid SNOUSSI

Formation : 2ème année BUT GEII – Électronique et Systèmes Embarqués

Période : 12 janvier 2026 – 6 mars 2026 (8 semaines)

Année universitaire : 2025-2026

Entreprise : EVIAS

Adresse : 2 Rue Gustave Eiffel, Bureau 4
10430 Rosières-près-Troyes, France

Maître de stage : Frederic GIL TIERNO

Tuteur de stage : Bastien JACQUOT

Table des matières

Remerciements	3
1 Introduction	4
2 Présentation de l'entreprise EVIAS	5
2.1 Historique et implantation	5
2.2 Effectif et organisation	6
2.3 Technologie employée	6
2.3.1 Le principe de la route électrique	6
2.3.2 Recharge statique et dynamique	7
2.3.3 L'infrastructure de charge	7
2.3.4 Avantages de la solution EVIAS	8
3 Problématique et présentation technique du sujet	9
3.1 Contexte et besoin	9
3.2 Cahier des charges	9
3.3 Architecture du système	10
3.3.1 Vue d'ensemble	10
3.3.2 Communication MQTT	10
3.3.3 Format des données	11
3.4 Matériel utilisé	12
3.4.1 Raspberry Pi	12
3.4.2 Écran tactile Waveshare 7.9"	12
4 Description du travail réalisé	13
4.1 Analyse de l'existant et choix techniques	13
4.1.1 Étude du code Python	13
4.1.2 Choix du protocole MQTT	13
4.1.3 Choix de Qt et C++	14
4.2 Développement de l'interface Qt/C++	14
4.2.1 Architecture du code	14
4.2.1.1 config.h : Configuration et énumérations	14
4.2.1.2 eidata.h : Structures de données	15
4.2.1.3 mqttclient.h/cpp : Client MQTT custom	15
4.2.1.4 mainwindow.h/cpp/ui : Interface principale	16
4.2.2 Fonctionnalités implémentées	17
4.2.2.1 Onglet MAIN : Vue opérateur	17
4.2.2.2 Onglet DETAILS : Informations complémentaires	18
4.2.2.3 Onglet DEBUG : Diagnostic technique	18
4.2.3 Calcul de la consommation énergétique	19
4.2.4 Gestion multi-véhicules	20

4.3	Intégration matérielle sur Raspberry Pi	20
4.3.1	Configuration de l'écran Waveshare	20
4.3.1.1	Résolution et orientation	20
4.3.1.2	Calibration tactile	20
4.3.1.3	Configuration Qt EGLFS	21
4.3.2	Splash screen au démarrage	21
4.3.2.1	Préparation de l'image	21
4.3.2.2	Service Systemd pour le splash screen	21
4.3.2.3	Masquage des logs de démarrage	22
4.3.3	Lancement automatique de l'interface	22
4.3.4	Gestion de la connectivité 4G/WiFi	23
4.3.5	Guide de déploiement	23
4.4	Difficultés rencontrées et solutions	24
4.4.1	Absence de QtMqtt dans les dépôts Raspberry Pi	24
4.4.2	Parsing JSON : int vs double	24
4.4.3	Configuration écran Waveshare	24
4.4.4	Conflit 4G/WiFi	24
4.4.5	Logs système au démarrage	24
5	Appréciation personnelle	25
5.1	Bilan technique	25
5.1.1	Compétences techniques développées	25
5.1.2	Méthodologie de travail	25
5.1.3	Défis relevés	26
5.2	Remerciements	26
	Glossaire	27
A	Annexes	28
A.1	Schémas techniques	30
A.1.1	Diagramme UML complet	30
A.1.2	Flux complet des données MQTT	31
A.1.3	Cycle de vie des données MQTT	32
A.2	Configuration Systemd	33
A.2.1	Service splash screen	33
A.2.2	Service application HMI	33
	English Summary	34

Remerciements

Je tiens à exprimer ma gratitude à toutes les personnes qui ont contribué à la réussite de ce stage et qui ont rendu ces huit semaines chez EVIAS enrichissantes.

Je remercie tout d'abord Manon Reynies, ingénieure en systèmes embarqués, pour son encadrement et la confiance qu'elle m'a accordée en me confiant ce projet stratégique pour l'entreprise. Je remercie également Frederic Gil Tierno, ingénieur en mécatronique, pour son suivi et ses recommandations durant cette expérience.

J'adresse aussi mes remerciements aux ingénieurs d'EVIAS pour leur accompagnement et leurs conseils, ainsi qu'à l'ensemble de l'équipe EVIAS pour leur accueil chaleureux et leur disponibilité.

Enfin, je remercie Bastien Jacquot, mon tuteur pédagogique, pour son suivi et ses conseils durant ce stage. Je remercie également l'IUT de Troyes pour la qualité de la formation dispensée, qui m'a permis d'aborder ce stage avec les compétences nécessaires.

Chapitre 1

Introduction

Le secteur du transport routier représente aujourd'hui l'une des principales sources d'émissions de CO₂ en France. Face à ce constat, l'électrification des véhicules s'impose comme une solution incontournable pour réduire notre empreinte carbone. Cependant, cette transition se heurte à plusieurs obstacles majeurs : l'autonomie limitée des batteries, les temps de recharge prolongés et le poids important des batteries nécessaires pour parcourir de longues distances.

EVIAS propose une solution innovante à ces problématiques : la recharge par conduction via des routes électriques. Cette technologie permet aux véhicules de se recharger en roulant (recharge dynamique) ou à l'arrêt (recharge statique) grâce à un rail électrique intégré dans la chaussée. Cette approche élimine le besoin de batteries volumineuses et réduit les temps d'arrêt pour recharge.

Dans le cadre de mon stage de deuxième année de BUT GEII option Électronique et Systèmes Embarqués, j'ai été chargé de développer une interface homme-machine (IHM) pour le monitoring de l'infrastructure de charge électrique (EI – EVIAS Infrastructure). L'objectif était de créer un système centralisé permettant de visualiser en temps réel l'ensemble des informations critiques de l'infrastructure : état des alimentations électriques, puissance délivrée et demandée, mode de charge, consommation énergétique, et état des différentes sections de rail.

Ce projet s'inscrit dans une phase de pré-industrialisation où la supervision et le monitoring de l'infrastructure deviennent essentiels pour assurer la fiabilité du système et faciliter sa maintenance. L'interface développée se destine à la fois aux opérateurs sur site et aux ingénieurs pour le diagnostic et l'optimisation du système.

Ce rapport présente le contexte de l'entreprise EVIAS, détaille le travail technique réalisé durant ces huit semaines, et analyse les compétences développées au cours de cette expérience professionnelle.

Chapitre 2

Présentation de l'entreprise EVIAS

2.1 Historique et implantation

EVIAS est une entreprise suédoise fondée en 2009 par Gunnar Asplund en réponse à un appel d'offres de la Suède visant à électrifier son réseau routier. Après plusieurs années de recherche et développement dans le domaine des routes électriques, EVIAS installe son premier prototype de route électrique près de Stockholm. Pendant plusieurs années, ce prototype est testé par le service postal local qui circule quotidiennement sur cette route entre l'aéroport et son entrepôt.

Au tournant de l'année 2021, la société est rachetée par Olivier Besson, un investisseur français, qui décide de la relocaliser en France. Un centre de recherche et développement est créé à Troyes, au sein du pôle technologique proche de l'Université de Technologie de Troyes (UTT). Une nouvelle route électrique est installée sur le campus de l'UTT et un second véhicule d'essai est acquis pour effectuer de nouveaux tests. En 2025, la phase de pré-industrialisation est lancée, marquant une étape importante dans le développement commercial de l'entreprise.



FIGURE 2.1 – Véhicule d'essai EVIAS en situation réelle

2.2 Effectif et organisation

EVIAS est actuellement composée d'une quinzaine de personnes réparties entre la France, la Suède et Hong Kong. Cette équipe internationale et multidisciplinaire rassemble des compétences variées : ingénieurs en systèmes embarqués, ingénieurs mécaniques, développeurs logiciels, chercheurs et personnel administratif.

L'entreprise fonctionne sur un modèle de startup agile où la communication et la collaboration sont essentielles. Les équipes françaises et suédoises travaillent en collaboration via des visioconférences régulières, permettant ainsi de partager l'expertise et d'accélérer le développement de la technologie.

2.3 Technologie employée

2.3.1 Le principe de la route électrique

Une route électrique, ou *e-road*, est une infrastructure permettant l'alimentation des véhicules électriques en circulation. Les technologies se répartissent en deux familles : la conduction (transfert d'énergie par contact physique) et l'induction (transfert sans contact). Les systèmes à caténaires, comparables à ceux utilisés pour les tramways, sont classés dans la famille de la conduction.

EVIAS a fait le choix de la technologie par conduction, jugée plus efficace en termes de rendement énergétique et plus économique à déployer à grande échelle. Le principe est simple : un rail électrique est intégré dans la chaussée et distribue du courant aux véhicules équipés d'un système de collecte qui se déploie sous le véhicule pour établir le contact électrique.

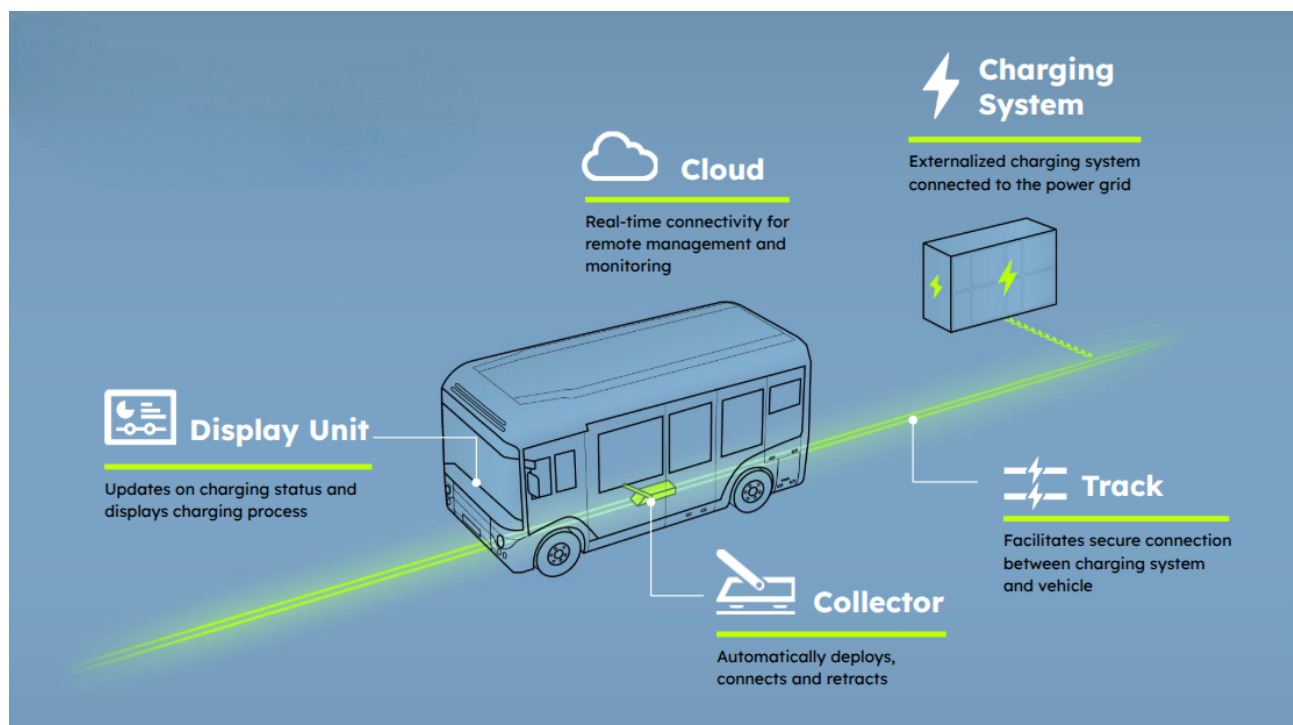


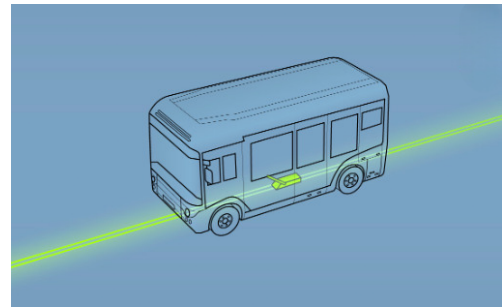
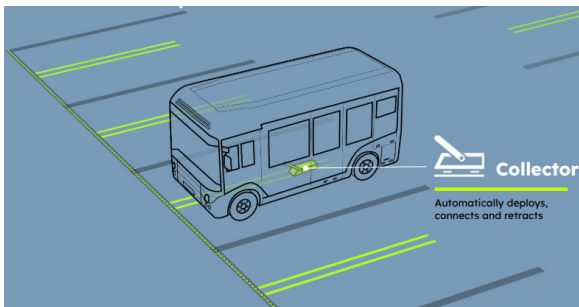
FIGURE 2.2 – Principe de fonctionnement de la technologie EVIAS

2.3.2 Recharge statique et dynamique

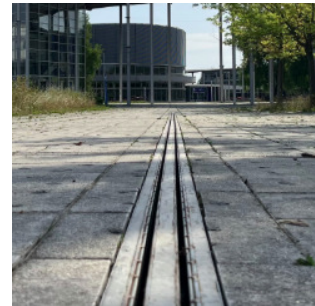
EVIAS propose deux modes de recharge par conduction :

La recharge statique : Le véhicule est à l'arrêt, par exemple sur une place de parking. Le rail peut être intégré directement dans le sol ou posé sur le sol au moyen d'un « bumper ». Ce mode est particulièrement adapté aux zones de stationnement, aux dépôts de bus ou aux aires de repos.

La recharge dynamique : Le véhicule est en mouvement et se connecte à une longue portion de rail électrique sur plusieurs kilomètres. Il se recharge ainsi tout en roulant, permettant de maintenir un niveau de charge constant sans nécessiter d'arrêt. Ce mode est idéal pour les axes routiers fréquentés et les trajets longue distance.



(a) Rail de recharge statique



(b) Rail de recharge dynamique

FIGURE 2.3 – Les deux modes de recharge proposés par EVIAS

2.3.3 L'infrastructure de charge

L'infrastructure de charge (EI – EVIAS Infrastructure), située à l'UTT, est le cœur du système. Elle comprend plusieurs composants essentiels :

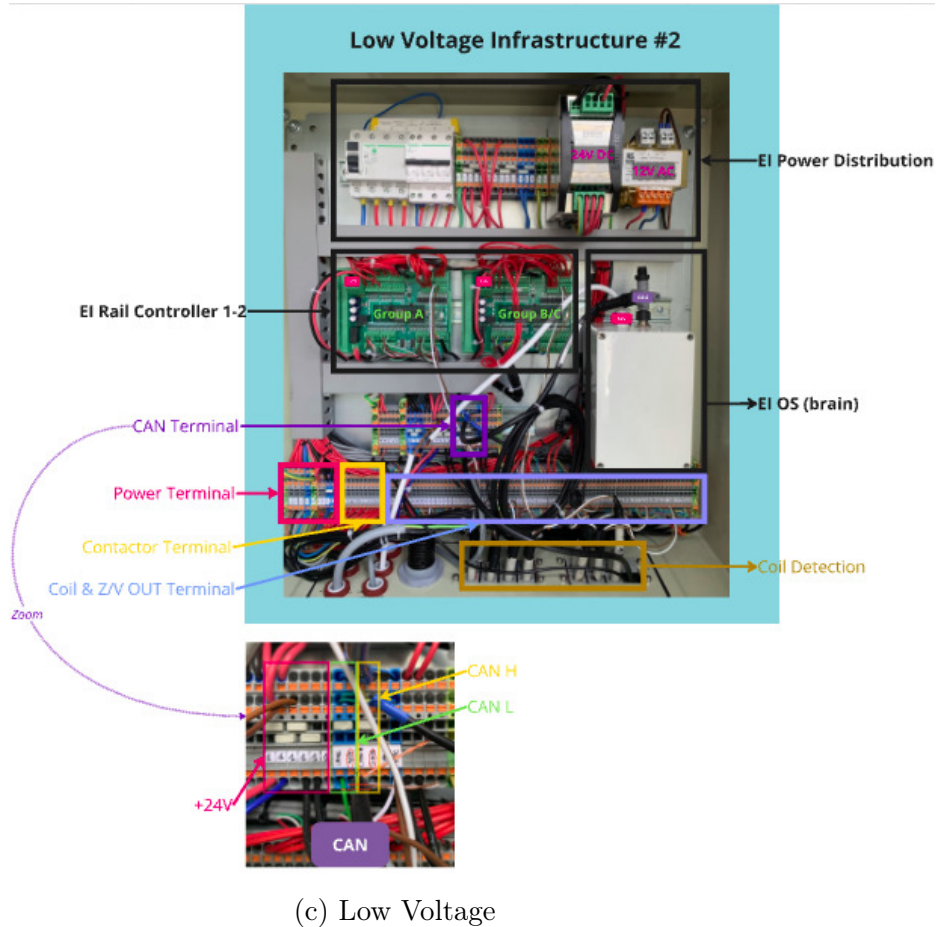
- **Les rails électriques** : intégrés dans la chaussée, ils acheminent l'électricité aux véhicules. Ils sont fractionnés en sections pour des raisons de sécurité (norme IP2x) et d'efficacité énergétique.
- **Les alimentations électriques (Power Supplies)** : au nombre de cinq dans l'installation de Troyes, elles convertissent et distribuent l'énergie électrique vers les rails. Chaque alimentation peut délivrer plusieurs dizaines de kilowatts.
- **Le système de contrôle (EI OS)** : c'est le cerveau de l'infrastructure. Il gère l'activation et la désactivation des sections de rail, communique avec les véhicules via MQTT, calcule la puissance à délivrer en fonction des besoins, et assure la facturation via le cloud EVIAS.
- **Le système de communication** : basé sur le protocole MQTT et le réseau 4G, il permet une communication temps réel entre l'infrastructure et les véhicules.



(a) Sous sol UTT



(b) High Voltage



(c) Low Voltage

FIGURE 2.4 – Infrastructure de charge électrique EVIAS UTT

2.3.4 Avantages de la solution EVIAS

Dans un contexte de transition énergétique et de lutte contre le réchauffement climatique, la technologie EVIAS présente plusieurs avantages :

- **Gain de temps** : les utilisateurs n'ont plus besoin de chercher des stations de recharge ni d'attendre pendant la recharge.
- **Réduction de la taille des batteries** : les véhicules n'ont besoin que de batteries d'appoint, réduisant ainsi le poids et la quantité de métaux lourds nécessaires.
- **Élimination du frein de l'autonomie** : la recharge dynamique permet de parcourir des distances illimitées sans contrainte d'autonomie.
- **Impact environnemental réduit** : batteries plus petites signifient moins d'extraction de métaux rares et moins de pollution lors de la fabrication et du recyclage.

Chapitre 3

Problématique et présentation technique du sujet

3.1 Contexte et besoin

Avant mon arrivée, l'infrastructure de charge EVIAS à Troyes fonctionnait de manière opérationnelle mais sans système de monitoring centralisé. Les ingénieurs et opérateurs ne disposaient pas d'une vision globale de l'état de l'infrastructure en temps réel.

Le besoin exprimé par l'équipe était clair : développer une interface permettant de visualiser l'ensemble des informations critiques de l'infrastructure depuis un point central. Cette interface devait afficher :

- L'état général de l'infrastructure (opérationnel, en erreur, sécurisé, etc.)
- La puissance délivrée par l'infrastructure aux véhicules
- La puissance demandée par les véhicules connectés
- L'état et la puissance de chaque alimentation électrique (5 Power Supplies)
- Le mode de charge actif (statique ou dynamique)
- La consommation énergétique totale
- L'état des différentes sections de rail
- Un historique local des événements
- Des pages techniques pour le diagnostic et le débogage

L'écran de l'interface devait être positionné dans la zone « Low Voltage Infrastructure (2.4) », à côté de la box EI OS qui contient la Raspberry Pi exécutant à la fois le code Python de gestion de l'infrastructure et le code Qt de l'interface graphique.

3.2 Cahier des charges

Le cahier des charges initial comprenait les fonctionnalités suivantes :

1. **Affichage de la puissance de charge en kWh** : visualisation de la puissance instantanée délivrée par l'infrastructure.
2. **Calcul de la consommation totale** : somme de l'énergie consommée depuis le démarrage ou le dernier reset, avec possibilité de réinitialisation.
3. **Mode statique/dynamique** : indication claire du mode de charge actif pour l'opérateur.
4. **Identification de l'infrastructure** : affichage de l'UUID de l'infrastructure, son nom et sa localisation.

5. **État de l'infrastructure** : visualisation de l'état opérationnel global (NOT_READY, SAFE, READY, ERROR).
6. **Pages techniques pour les ingénieurs** : accès aux données système brutes, messages MQTT, informations de diagnostic.
7. **Historique local** : enregistrement et affichage des événements récents pour faciliter le débogage et la maintenance.

Au-delà de ce cahier des charges, des fonctionnalités complémentaires ont été ajoutées en cours de développement suite aux échanges avec les ingénieurs : affichage détaillé des 5 Power Supplies, visualisation de l'état des 14 sections de rail, séparation des données de puissance en deux colonnes (puissance fournie vs puissance demandée), et gestion multi-véhicules.

3.3 Architecture du système

3.3.1 Vue d'ensemble

Le système de monitoring s'insère dans une architecture distribuée comprenant plusieurs composants communiquant via le protocole MQTT (voir détail du flux complet en annexe A.3, page 31) :

- **Le code Python EI OS** : installé sur la Raspberry Pi, il gère la communication avec le bus CAN, et publie les informations d'état sur le broker MQTT.
- **Le broker MQTT** : serveur Mosquitto qui gère la distribution des messages entre les différents composants (infrastructure, véhicules, interface).
- **L'interface Qt/C++** : application développée durant ce stage, elle s'abonne aux topics MQTT pertinents, reçoit les données en temps réel, et affiche les informations de manière claire et organisée.

3.3.2 Communication MQTT

Le protocole MQTT (Message Queuing Telemetry Transport) a été choisi pour sa légèreté et son efficacité dans les environnements à ressources limitées. Il fonctionne selon un modèle publication/abonnement où les clients (infrastructure, véhicules, interface) publient des messages sur des *topics* et s'abonnent aux topics qui les intéressent (voir structure cycle de donnée MQTT en annexe A.4, page 32)

Les topics MQTT utilisés suivent une hiérarchie claire :

Listing 3.1 – Topics MQTT de l'infrastructure

```

1 EVIAS/ei/1/ei_to_eck/status          # Etat de l'infrastructure
2 EVIAS/ei_os/intern/power_supply/data # Donnees des alimentations

```

Listing 3.2 – Topics MQTT des vehicules

```

1 EVIAS/vehicle/+/eck_os/eck_to_ei/status          # Etat vehicule
2 EVIAS/vehicle/+/eck_os/eck_to_cloud/vehicle/charging_mode # Mode
  charge
3 EVIAS/vehicle/+/eck_os/eck_to_cloud/vcu/power_request      # Demande
  puissance

```

Le symbole + dans les topics est un *wildcard* qui permet de s'abonner à tous les véhicules simultanément, quelle que soit leur ID. Cette approche facilite la gestion multi-véhicules sans modifier le code.

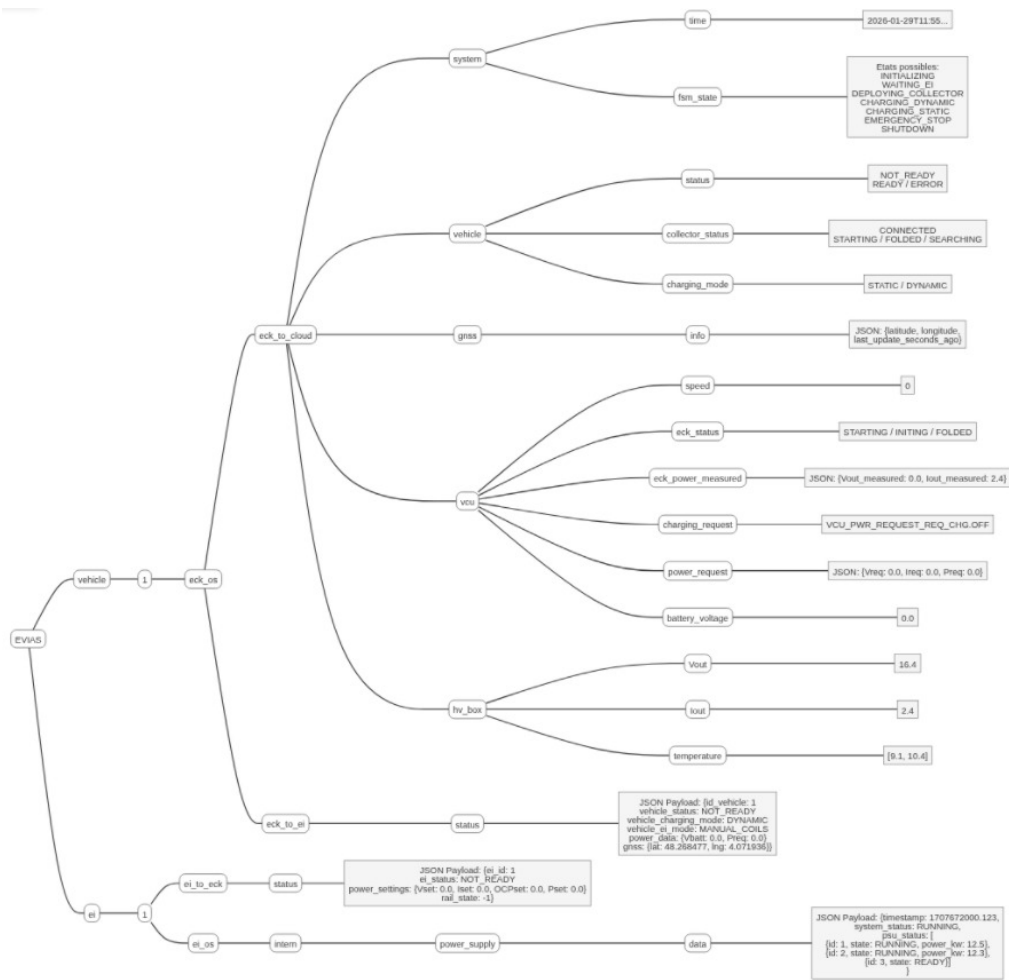


FIGURE 3.1 – Structure complète des topics MQTT (schéma Miro)

3.3.3 Format des données

Les messages MQTT transportent des données au format JSON. Voici quelques exemples de structures utilisées :

Listing 3.3 – Message d'état de l'infrastructure

```

1 {
2   "ei_id": "88eeb872-80c7-483c-a7e9-b9455a5098d7",
3   "ei_status": "READY",
4   "power_settings": {
5     "Vset": 630.0,
6     "Iset": 45.0,
7     "Pset": 28.35
8   },
9   "rail_state": "5"
10 }

```

Listing 3.4 – Message des alimentations électriques

```

1 {
2   "timestamp": 1707672000.123,
3   "system_state": "RUNNING",
4   "psu_status": [
5     {"id": 1, "state": "RUNNING", "power_kw": 12.5},

```



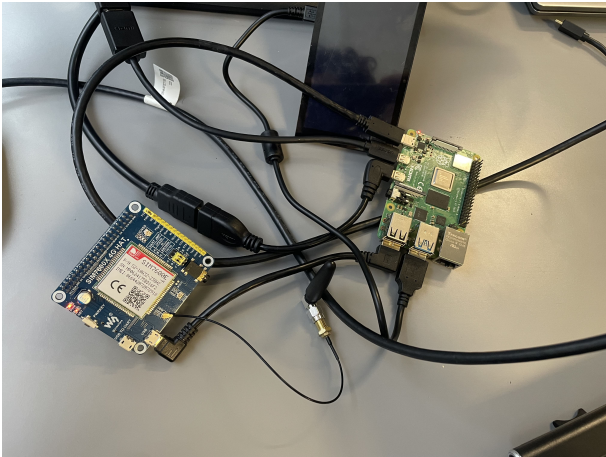
```
6     {"id": 2, "state": "RUNNING", "power_kw": 8.3},  
7     {"id": 3, "state": "READY"}  
8 ]  
9 }
```

3.4 Matériel utilisé

3.4.1 Raspberry Pi

Le cœur du système est une Raspberry Pi 4 Model B équipée d'un processeur ARM Cortex-A72 quad-core à 1.8 GHz et de 4 Go de RAM. Elle exécute Raspberry Pi OS (basé sur Debian) et héberge à la fois le code Python de gestion de l'infrastructure et l'interface Qt/C++ développée durant ce stage.

En fin de projet, le système a été migré vers une Raspberry Pi industrielle intégrant tous les composants sur une seule carte (Raspberry Pi, driver CAN, module 4G), offrant ainsi une solution plus robuste et professionnelle pour un déploiement en conditions réelles.



(a) Setup de développement



(b) Version finale industrielle

FIGURE 3.2 – Évolution du matériel

3.4.2 Écran tactile Waveshare 7.9"

L'interface est affichée sur un écran tactile Waveshare de 7.9 pouces avec une résolution atypique de 400×1280 pixels (voir position dans l'infrastructure en annexe [A.1](#), page 29). Cet écran capacitif offre une bonne réactivité tactile et une luminosité suffisante pour une utilisation en extérieur.

La configuration de cet écran a nécessité un travail spécifique : calibration de la matrice tactile avec la bibliothèque TSlib, configuration du système d'affichage Qt EGLFS (Embedded GL), et ajustement de la rotation de l'écran (-90°) pour correspondre à son orientation physique.

Chapitre 4

Description du travail réalisé

Ce chapitre détaille l'ensemble du travail technique effectué durant ces huit semaines de stage. Il est organisé selon les grandes phases du projet : analyse de l'existant, développement de l'interface, intégration matérielle, et documentation.

4.1 Analyse de l'existant et choix techniques

4.1.1 Étude du code Python

La première étape a consisté à comprendre l'architecture logicielle existante. Le système EI OS est composé de deux programmes Python principaux :

- **EI_OS_CAN_HANDLER** : gère la communication avec le bus CAN du véhicule, lit les messages des différents capteurs et actionneurs, et publie ces informations sur le broker MQTT interne.
- **EI_OS_STATIC** : implémente la machine à états (FSM – Finite State Machine) qui pilote l'infrastructure, gère les Power Supplies, contrôle l'activation des sections de rail, et publie l'état de l'infrastructure sur MQTT.

L'analyse de ces programmes a permis d'identifier les topics MQTT pertinents pour l'interface, la structure des messages JSON échangés, et les différents états possibles du système.

4.1.2 Choix du protocole MQTT

Initialement, j'ai étudié la possibilité d'accéder directement au bus CAN depuis l'interface Qt. Cependant, cette approche présentait plusieurs inconvénients :

- Complexité accrue : nécessité de parser les messages CAN, de gérer les différents identifiants, et de maintenir la cohérence avec le code Python.
- Couplage fort : modification difficile de l'architecture système.
- Redondance : duplication de la logique déjà implémentée en Python.

Le choix du protocole MQTT s'est imposé comme une solution plus élégante :

- Architecture découplée : l'interface s'abonne simplement aux topics pertinents sans connaître les détails du bus CAN.
- Maintenance facilitée : les modifications du code Python n'impactent pas l'interface tant que la structure des messages JSON reste stable.
- Évolutivité : possibilité d'ajouter facilement d'autres clients (applications mobiles, dashboards web, etc.).

4.1.3 Choix de Qt et C++

Pour le développement de l'interface, j'ai choisi le framework Qt avec le langage C++ pour plusieurs raisons :

- **Performance** : Qt est reconnu pour son efficacité sur les systèmes embarqués comme la Raspberry Pi.
- **Support natif Linux** : excellente intégration avec les systèmes Linux sans nécessiter de serveur X.
- **Écran tactile** : support natif du tactile via le plugin EGLFS.
- **Expérience** : framework utilisé durant ma formation.

Le choix entre Qt Widgets et QML s'est porté sur Qt Widgets pour sa simplicité et ses performances dans notre cas d'usage.

4.2 Développement de l'interface Qt/C++

4.2.1 Architecture du code

Le code de l'interface a été structuré de manière modulaire pour faciliter la maintenance et les évolutions futures (voir diagramme UML complet en annexe A.2, page 30) :

Listing 4.1 – Structure des fichiers du projet

```

1 EVIAS_EI_HMI /
2 - config.h           # Constantes, enums, couleurs officiel
3 - eidata.h           # Structures de donnees
4 - mqttclient.h/cpp   # Client MQTT custom
5 - mainwindow.h/cpp/ui # Interface principale
6 - main.cpp           # Point d'entree
7 - CMakeLists.txt     # Configuration compilation

```

4.2.1.1 config.h : Configuration et énumérations

Ce fichier centralise toutes les constantes du projet : adresses du broker MQTT, identifiants, topics, limites physiques, et couleurs de la charte graphique EVIAS.

Le fichier définit également les énumérations pour les différents états du système :

Listing 4.2 – Extrait de config.h - Enumerations

```

1 enum class EIStatus {
2     NOT_READY,
3     SAFE,
4     READY,
5     ERROR,
6     UNKNOWN
7 };
8
9 enum class ChargingMode {
10     STATIC,
11     DYNAMIC,
12     UNKNOWN
13 };
14
15 enum class PSSState {

```

```

16     STANDING_BY,
17     INITING,
18     PRECHARGING,
19     READY,
20     RUNNING,
21     ERROR,
22     UNKNOWN
23 };

```

4.2.1.2 eidata.h : Structures de données

Ce fichier header-only définit les structures de données utilisées pour stocker les informations de l'infrastructure et des véhicules :

Listing 4.3 – Extrait de eidata.h - Structures principales

```

1 struct PowerSettings {
2     double voltage;    // Vset (V)
3     double current;    // Iset (A)
4     double power;      // Pset (kW)
5 };
6
7 struct PowerRequest {
8     double voltageReq; // Vreq (V)
9     double currentReq; // Ireq (A)
10    double powerReq;    // Preq (kW)
11 };
12
13 struct PowerSupplyData {
14     int id;             // 1 à 5
15     PSSState state;     // Etat (RUNNING, READY, ...)
16     double power_kw;    // Puissance en kW
17     double vout;        // Tension sortie (V)
18     double iout;        // Courant sortie (A)
19 };

```

La classe `EIData` centralise toutes ces données et fournit des méthodes pour les mettre à jour à partir des messages JSON reçus via MQTT.

4.2.1.3 mqttclient.h/cpp : Client MQTT custom

L'une des difficultés majeures du projet a été l'absence de la bibliothèque `QtMqtt` dans les dépôts officiels de Raspberry Pi OS. Plutôt que de compiler cette bibliothèque depuis les sources (opération longue), j'ai développé un client MQTT custom en utilisant uniquement les sockets TCP de Qt.

Le client implémente le protocole MQTT au niveau bas :

- Construction manuelle des packets CONNECT, SUBSCRIBE, PUBLISH
- Parsing des packets CONNACK, PUBLISH reçus
- Gestion du keep-alive avec envoi périodique de PING
- Authentification par nom d'utilisateur et mot de passe

Listing 4.4 – Extrait de mqttclient.cpp - Connexion au broker

```

1 void MQTTClient::connectToBroker(const QString &host, quint16 port)

```

```

2 {
3     m_host = host;
4     m_port = port;
5     qDebug() << "Connecting to MQTT broker:" << host << ":" << port;
6     m_socket->connectToHost(host, port);
7 }
8
9 void MQTTClient::onConnected()
10 {
11     qDebug() << "TCP connection established. Sending CONNECT packet..."
12     ;
13     sendConnectPacket();
14 }

```

Le parsing des messages PUBLISH entrants extrait le topic et le payload JSON :

Listing 4.5 – Extrait de mqttclient.cpp - Parsing PUBLISH

```

1 void MQTTClient::parseMessage(const QByteArray &data)
2 {
3     quint8 messageType = (data[0] >> 4) & 0x0F;
4
5     if (messageType == 3) { // PUBLISH
6         // Extraire la longueur du topic
7         quint16 topicLen = (data[pos] << 8) | data[pos + 1];
8         pos += 2;
9
10        // Extraire le topic
11        QString topic = QString::fromUtf8(data.mid(pos, topicLen));
12        pos += topicLen;
13
14        // Extraire le payload
15        QByteArray payload = data.mid(pos, remainingLength - topicLen -
16            2);
17
18        // Parser le JSON
19        QJsonDocument doc = QJsonDocument::fromJson(payload);
20        if (doc.isObject()) {
21            emit messageReceived(topic, doc.object());
22        }
23    }
24 }

```

4.2.1.4 mainwindow.h/cpp/ui : Interface principale

La classe MainWindow gère l'interface graphique et la logique d'affichage. Elle s'abonne aux topics MQTT pertinents, reçoit les messages via des signaux Qt, met à jour les structures de données, et rafraîchit l'affichage.

L'interface est organisée en trois onglets :

1. **MAIN** : Vue opérateur avec les informations essentielles
2. **DETAILS** : Informations complémentaires et historique des événements
3. **DEBUG** : JSON brut et logs pour le diagnostic

Listing 4.6 – Extrait de mainwindow.cpp - Gestion messages MQTT

```

1 void MainWindow::onMqttMessageReceived(const QString &topic, const
  QJsonObject &payload)
2 {
3     // Topic EI status
4     if (topic.contains("ei_to_eck/status")) {
5         eiData.updateFromJson(payload);
6         refreshUI();
7     }
8     // Topic ECK power request
9     else if (topic.contains("vcu/power_request")) {
10        eiData.updatePowerRequestFromJson(payload);
11        updatePowerRequest(eiData.getPowerRequest());
12    }
13    // Topic Power Supplies
14    else if (topic.contains("power_supply/data")) {
15        QJsonArray psuArray = payload.value("psu_status").toArray();
16        for (const QJsonValue &psuValue : psuArray) {
17            QJsonObject psuObj = psuValue.toObject();
18            int psId = psuObj.value("id").toInt();
19            eiData.updatePowerSupply(psId, psuObj);
20        }
21        updatePowerSupplies();
22    }
23 }

```

4.2.2 Fonctionnalités implémentées

4.2.2.1 Onglet MAIN : Vue opérateur

L'onglet principal affiche en temps réel les informations critiques de l'infrastructure :

Bloc État EI :

- État actuel de l'infrastructure (NOT_READY, SAFE, READY, ERROR)
- Code couleur clair : rouge pour erreur, vert pour SAFE, bleu-gris pour READY
- UUID de l'infrastructure pour identification
- Mode de charge actif (STATIC ou DYNAMIC) avec code couleur

Bloc Données de Puissance (2 colonnes) :

- *Colonne gauche – EI Settings* : puissance délivrée par l'infrastructure (Vset, Iset, Pset)
- *Colonne droite – ECK Request* : puissance demandée par le véhicule (Vreq, Ireq, Preq)
- Barre de progression visuelle de la puissance
- Calcul et affichage de la consommation totale en kWh avec bouton de réinitialisation

Bloc Power Supplies :

- Affichage de l'état des 5 alimentations électriques
- Code couleur par état (RUNNING = vert, READY = bleu, STANDBY = gris, ERROR = rouge)
- Puissance instantanée en kW pour chaque alimentation active

Bloc Rail State :

- Visualisation des 14 sections de rail
- Section active colorée en vert, sections inactives en gris
- Mise à jour en temps réel selon le rail utilisé

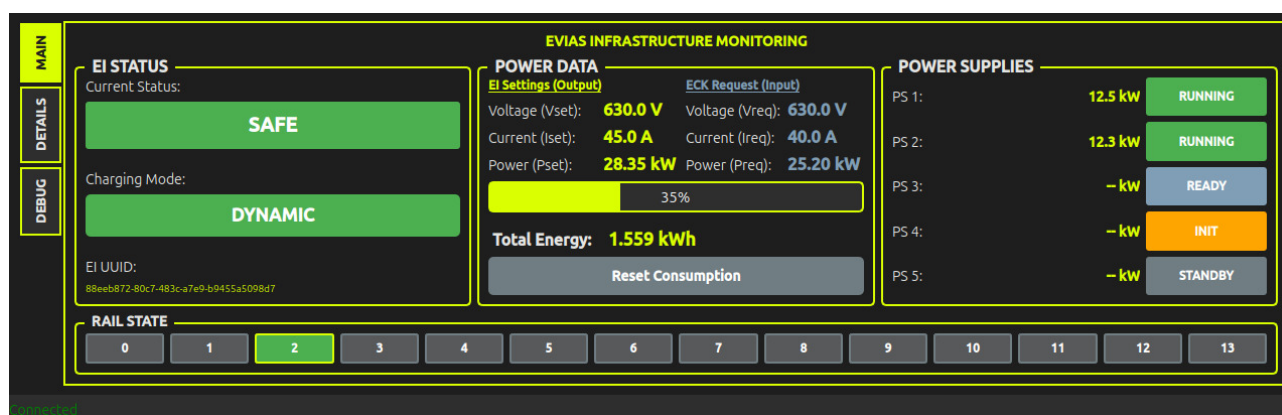


FIGURE 4.1 – Onglet MAIN V5.2 – Vue opérateur

4.2.2.2 Onglet DETAILS : Informations complémentaires

L'onglet DETAILS fournit des informations additionnelles utiles pour la maintenance et le diagnostic :

- Nom et emplacement de l'infrastructure
- Date et heure de la dernière mise à jour des données
- Historique local des événements récents (connexion MQTT, changements d'état, erreurs)
- Limitation automatique à 100 lignes d'historique pour éviter la saturation mémoire

L'historique utilise un affichage formaté en HTML pour une meilleure lisibilité avec horodatage et code couleur selon le type d'événement.

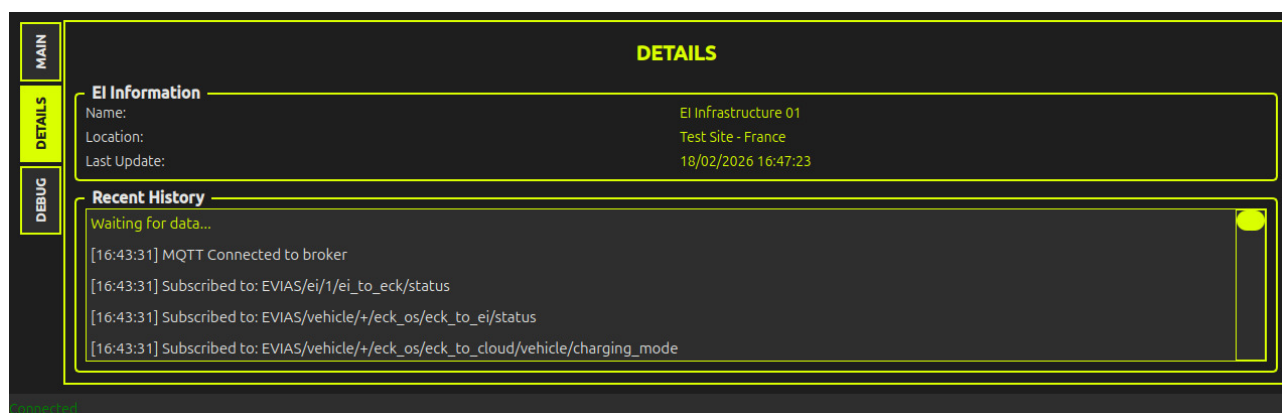


FIGURE 4.2 – Onglet DETAILS V5.0 – Historique et informations

4.2.2.3 Onglet DEBUG : Diagnostic technique

L'onglet DEBUG est destiné aux ingénieurs et développeurs pour le diagnostic :

- Affichage du JSON brut des derniers messages MQTT reçus
- Informations de connexion au broker MQTT (hôte, port, topics)
- Bouton pour effacer le log et libérer la mémoire
- Bouton "Start DEMO" pour démarré une simulation du fonctionnement de l'application

Cet onglet est particulièrement utile durant la phase de développement pour identifier les problèmes de communication ou de parsing.

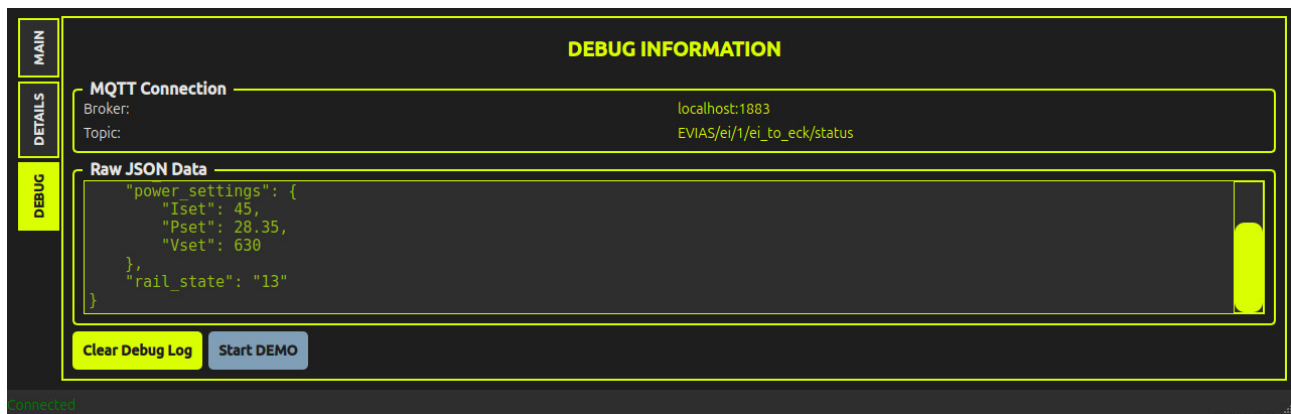


FIGURE 4.3 – Onglet DEBUG V5.0 – Données brutes et diagnostic

4.2.3 Calcul de la consommation énergétique

Une des fonctionnalités clés demandées était le calcul de la consommation énergétique totale. L'implémentation utilise une intégration numérique simple de la puissance dans le temps :

$$E_{total} = \sum_{i=1}^n P_i \times \Delta t_i \quad (4.1)$$

où P_i est la puissance instantanée en kW et Δt_i l'intervalle de temps en heures.

Listing 4.7 – Extrait de mainwindow.cpp - Calcul consommation

```

1 void MainWindow::updateConsumption()
2 {
3     if (!eiData.isDataFresh(10)) {
4         return; // Ne pas calculer si donnees obsoletes
5     }
6
7     QDateTime now = QDateTime::currentDateTime();
8     double timeElapsedSeconds = lastConsumptionUpdate.msecsTo(now) /
9         1000.0;
10    double timeElapsedHours = timeElapsedSeconds / 3600.0;
11
12    // Puissance actuelle en kW
13    double currentPowerKW = eiData.getPowerSettings().getPowerKW();
14
15    // Energie pour cet intervalle
16    double energyDeltaKWh = currentPowerKW * timeElapsedHours;
17
18    // Ajout au total
19    totalEnergyKWh += energyDeltaKWh;
20
21    // Mise a jour affichage
22    ui->label_consumption_value->setText(
23        QString::number(totalEnergyKWh, 'f', 3) + " kWh"
24    );
25
26    lastConsumptionUpdate = now;
27 }

```


Un timer Qt déclenche cette fonction toutes les secondes pour maintenir le calcul à jour. Un bouton permet de réinitialiser le compteur à zéro.

4.2.4 Gestion multi-véhicules

Pour supporter plusieurs véhicules simultanément sans modification du code, l'interface utilise des wildcards dans les topics MQTT. Le symbole + remplace l'ID du véhicule :

Listing 4.8 – Topics avec wildcards

```
1 EVIAS/vehicle/+/eck_os/eck_to_ei/status
2 EVIAS/vehicle/+/eck_os/eck_to_cloud/vehicle/charging_mode
3 EVIAS/vehicle/+/eck_os/eck_to_cloud/vcu/power_request
```

Lors de la réception d'un message, le code utilise `contains()` au lieu de comparaisons d'égalité strictes pour matcher les topics :

Listing 4.9 – Matching de topics avec wildcards

```
1 if (topic.contains("eck_to_ei/status")) {
2     // Traiter le message quel que soit l'ID du vehicule
3 }
```

Cette approche simple et efficace permet de gérer dynamiquement plusieurs véhicules sans configuration supplémentaire.

4.3 Intégration matérielle sur Raspberry Pi

4.3.1 Configuration de l'écran Waveshare

L'écran Waveshare 7.9" présente plusieurs particularités nécessitant une configuration spécifique :

4.3.1.1 Résolution et orientation

La résolution native de l'écran est 400×1280 pixels en orientation portrait. Pour une utilisation plus ergonomique, l'écran a été monté en orientation paysage, nécessitant une rotation de -90° appliquée au niveau du système Qt via la variable d'environnement :

```
1 export QT_QPA_EGLFS_ROTATION=-90
```

4.3.1.2 Calibration tactile

Le tactile capacitif nécessite une calibration précise pour que les coordonnées tactiles correspondent aux positions à l'écran. Cette calibration a été effectuée avec la bibliothèque TSlib :

Listing 4.10 – Processus de calibration

```
1 # Installation de TSlib
2 sudo apt install tslib
3
4 # Calibration interactive
5 sudo ts_calibrate
6
7 # Application de la calibration
```

```

8 export TSLIB_TSDEVICE=/dev/input/by-id/usb-WaveShare_WaveShare_
    000000000089-event-if00

```

La bibliothèque Tslib génère un fichier de calibration contenant la matrice de transformation entre les coordonnées brutes du tactile et les coordonnées écran.

4.3.1.3 Configuration Qt EGLFS

Qt a été configuré pour utiliser le plugin EGLFS (Embedded GL Full Screen) qui permet un affichage direct sur le framebuffer sans serveur X11, optimisant ainsi les performances :

Listing 4.11 – Variables d’environnement Qt

```

1 export QT_QPA_PLATFORM=eglfs
2 export QT_QPA_EGLFS_ROTATION=-90
3 export QT_QPA_EGLFS_HIDE_CURSOR=1
4 export QT_QPA_GENERIC_PLUGINS=tslib
5 export TSLIB_TSDEVICE=/dev/input/by-id/usb-WaveShare_WaveShare_
    000000000089-event-if00

```

4.3.2 Splash screen au démarrage

Pour offrir une expérience utilisateur professionnelle, un écran de démarrage affichant le logo EVIAS a été mis en place. Plutôt que d’utiliser Plymouth (qui gère mal les résolutions atypiques), j’ai opté pour FBI (Linux Framebuffer Imageviewer) :

4.3.2.1 Préparation de l’image

L’image du logo EVIAS a été redimensionnée aux dimensions de l’écran (1280×400 en orientation paysage) au format PNG.

4.3.2.2 Service Systemd pour le splash screen

Un service Systemd dédié affiche le logo dès le démarrage du système :

Listing 4.12 – /etc/systemd/system/splashscreen.service

```

1 [Unit]
2 Description= Splash Screen EVIAS
3 DefaultDependencies=no
4 After=local-fs.target
5
6 [Service]
7 ExecStartPre=/usr/bin/chvt 1
8 ExecStart=/usr/bin/fbi -d /dev/fb0 --noverbose -a /home/evias/splash.
    png
9 StandardInput=tty
10 StandardOutput=tty
11 TTYPath=/dev/tty1
12 Type=oneshot
13
14 [Install]
15 WantedBy=sysinit.target

```

4.3.2.3 Masquage des logs de démarrage

Pour éviter que les logs Linux n'apparaissent pendant le démarrage, le noyau a été configuré pour rediriger la console et supprimer les messages de démarrage :

Listing 4.13 – Modification de /boot/cmdline.txt

```
1 console=tty3 quiet loglevel=0 logo.nologo vt.global_cursor_default=0
```



FIGURE 4.4 – Écran de démarrage avec logo EVIAS

4.3.3 Lancement automatique de l'interface

L'interface doit se lancer automatiquement au démarrage de la Raspberry Pi sans intervention manuelle. Un service Systemd a été créé pour gérer ce lancement :

Listing 4.14 – /etc/systemd/system/evias_ei_hmi.service

```
1 [Unit]
2 Description=EVIAS HMI Application
3 Conflicts=splashscreen.service
4 After=local-fs.target
5 Before=getty@tty1.service
6
7 [Service]
8 User=root
9 WorkingDirectory=/home/evias/EVIAS_EI_HMI/build
10
11 # Variables d'environnement
12 Environment=QT_QPA_PLATFORM=eglfs
13 Environment=QT_QPA_EGLFS_ROTATION=-90
14 Environment=QT_QPA_EGLFS_HIDECURSOR=1
15 Environment=QT_QPA_GENERIC_PLUGINS=tslib
16 Environment=TSLIB_TSDEVICE=/dev/input/by-id/usb-WaveShare_WaveShare...
17 Environment=QT_QPA_PLATFORMTHEME=qt6ct
18 Environment=QT_STYLE_OVERRIDE=fusion
19
20 # Lancement de l'application
21 ExecStart=/home/evias/EVIAS_EI_HMI/build/EVIAS_EI_HMI
```

```

22
23 StandardInput=tty
24 StandardOutput=tty
25 TTYPath=/dev/tty1
26 Type=simple
27
28 # Redemarrage automatique en cas de crash
29 Restart=always
30 RestartSec=2
31
32 [Install]
33 WantedBy=multi-user.target

```

Le service est configuré pour :

- S'exécuter en tant que root (nécessaire pour l'accès direct au framebuffer)
- Entrer en conflit avec le service splash screen (transition propre entre le logo et l'interface)
- Redémarrer automatiquement en cas de crash
- Attendre que le système de fichiers local soit prêt avant de démarrer

Activation du service :

```

1 sudo systemctl enable evias_ei_hmi.service
2 sudo systemctl start evias_ei_hmi.service

```

4.3.4 Gestion de la connectivité 4G/WiFi

La Raspberry Pi est équipée d'un module 4G (SIM7600) connecté via USB pour assurer la communication avec le broker MQTT distant. La coexistence du module 4G et du WiFi intégré a nécessité une configuration spécifique de NetworkManager pour gérer les priorités de routage.

Sans configuration, le système privilégiait systématiquement l'interface 4G (métrique plus basse), même lorsqu'aucune carte SIM n'était insérée, coupant ainsi l'accès WiFi nécessaire pour le développement.

La solution a consisté à ajuster les métriques de routage via NetworkManager :

Listing 4.15 – Configuration des priorites reseau

```

1 # Verification des routes
2 ip route show
3
4 # Ajustement des metriques dans NetworkManager
5 sudo nmcli connection modify <wifi-connection> ipv4.route-metric 100
6 sudo nmcli connection modify <4g-connection> ipv4.route-metric 200
7
8 # Desactivation de la mise en veille WiFi
9 sudo nmcli connection modify <wifi-connection> wifi.powersave 2

```

Cette configuration garantit que le WiFi est privilégié lorsqu'il est disponible (développement en local), tandis que la 4G prend le relais automatiquement lorsque le système est déployé sur site sans WiFi.

4.3.5 Guide de déploiement

Un document récapitulatif des étapes de déploiement sur une nouvelle Raspberry Pi a été rédigé :

1. Installation de Raspberry Pi OS
2. Installation des dépendances (Qt, TSlib, FBI, Mosquitto)
3. Configuration de l'écran Waveshare (résolution, tactile)
4. Compilation du projet
5. Configuration des services Systemd
6. Configuration réseau (WiFi, 4G)
7. Tests et validation

4.4 Difficultés rencontrées et solutions

4.4.1 Absence de QtMqtt dans les dépôts Raspberry Pi

Problème : La bibliothèque QtMqtt n'est pas disponible dans les dépôts officiels de Raspberry Pi OS, et sa compilation depuis les sources est longue et complexe.

Solution : Développement d'un client MQTT custom en utilisant uniquement les sockets TCP de Qt.

4.4.2 Parsing JSON : int vs double

Problème : Certaines valeurs numériques dans les messages JSON étaient parfois reçues comme entiers et parfois comme flottants selon la source, provoquant des erreurs de parsing.

Solution : Utilisation systématique de `.toVariant().toDouble()` au lieu de `.toDouble()` directement, permettant une conversion automatique quelle que soit le type d'origine dans le JSON.

4.4.3 Configuration écran Waveshare

Problème : L'écran Waveshare ne s'affichait pas correctement avec les paramètres par défaut de Qt. La résolution atypique (400×1280) et la nécessité de rotation posaient des difficultés.

Solution : Configuration spécifique de Qt EGLFS via les variables d'environnement, calibration manuelle du tactile avec TSlib, et identification précise du périphérique tactile via `/dev/input/by-id/` pour éviter les changements aléatoires au redémarrage.

4.4.4 Conflit 4G/WiFi

Problème : Le système privilégiait systématiquement l'interface 4G (même sans carte SIM), coupant l'accès WiFi nécessaire pour le développement.

Solution : Ajustement des métriques de routage dans NetworkManager pour donner la priorité au WiFi, avec bascule automatique sur 4G lorsque le WiFi n'est pas disponible.

4.4.5 Logs système au démarrage

Problème : Les logs de démarrage de Linux s'affichaient à l'écran avant le lancement de l'interface, donnant une impression peu professionnelle.

Solution : Mise en place d'un splash screen avec FBI, redirection de la console vers `tty3`, et suppression des messages de boot via les paramètres du noyau dans `/boot/cmdline.txt`.

Chapitre 5

Appréciation personnelle

5.1 Bilan technique

Ce stage de huit semaines chez EVIAS a été une expérience enrichissante sur le plan technique. J'ai pu mettre en pratique et approfondir de nombreuses compétences acquises durant ma formation en BUT GEII, tout en découvrant de nouveaux domaines.

5.1.1 Compétences techniques développées

Développement Qt/C++ : J'ai pu découvrir des spécificités du développement pour systèmes embarqués, notamment l'utilisation du plugin EGLFS et la gestion du tactile sans serveur X11.

Protocoles de communication : L'implémentation d'un client MQTT depuis les sockets TCP bruts m'a donné une bonne compréhension du protocole. Cette expérience m'a appris l'importance de maîtriser les couches basses des protocoles de communication, au-delà de l'utilisation de bibliothèques haut niveau.

Linux embarqué : La configuration de la Raspberry Pi, la gestion des services Systemd, la calibration des périphériques, et l'optimisation du démarrage m'ont initié au monde des systèmes embarqués Linux.

Parsing et traitement de données JSON : La manipulation de structures de données et leur transformation en informations visuelles claires m'a montré les problématiques de monitoring temps réel.

5.1.2 Méthodologie de travail

Au-delà des aspects purement techniques, ce stage m'a permis de développer une méthodologie de travail :

- **Analyse avant action** : Prendre le temps de bien comprendre l'architecture existante avant de commencer à coder s'est révélé essentiel pour faire les bons choix techniques.
- **Développement itératif** : Plutôt que de vouloir tout implémenter d'un coup, j'ai adopté une approche par fonctionnalités successives, permettant de tester et valider chaque partie indépendamment.
- **Documentation continue** : Documenter le code au fur et à mesure plutôt qu'à la fin a facilité la maintenance et les évolutions.
- **Communication régulière** : Les réunions hebdomadaires avec les ingénieurs ont permis d'ajouter des fonctionnalités pertinentes au fil du projet.

5.1.3 Défis relevés

Plusieurs défis techniques ont marqué ce stage :

- Le développement d'un client MQTT custom sans bibliothèque existante a été un défi qui m'a permis de comprendre les différentes étapes de ce protocole.
- L'intégration de tous les composants (Qt, MQTT, écran, Systemd, réseau) dans un système cohérent a nécessité une vision globale et une capacité à déboguer à différents niveaux (application, système, réseau).

5.2 Remerciements

Je tiens à remercier toute l'équipe EVIAS pour cette expérience formatrice. La confiance accordée, l'autonomie donnée m'ont permis de mener ce projet à bien et de progresser sur le plan professionnel.

Glossaire

- Bus CAN (Controller Area Network)** Protocole de communication série utilisé dans l'automobile pour connecter les différents calculateurs et capteurs d'un véhicule.
- Broker MQTT** Serveur intermédiaire qui gère la distribution des messages entre les clients MQTT (publishers et subscribers).
- ECK (EVIAS Collector Kit)** Système embarqué dans les véhicules permettant la connexion automatique aux rails électriques pour la recharge.
- EGLFS (Embedded GL Full Screen)** Plugin Qt permettant un rendu graphique direct sur le framebuffer Linux sans serveur X11, optimisé pour les systèmes embarqués.
- EI (EVIAS Infrastructure)** Ensemble de l'infrastructure de charge électrique : rails, alimentations, système de contrôle.
- EI OS** Logiciel de gestion de l'infrastructure EVIAS, écrit en Python, qui contrôle les alimentations et les rails.
- Framebuffer** Zone de mémoire contenant les données d'affichage d'un écran sous Linux, accessible directement sans serveur graphique.
- FSM (Finite State Machine)** Machine à états finis, modèle de contrôle utilisé pour gérer les différents états de l'infrastructure.
- JSON (JavaScript Object Notation)** Format léger d'échange de données structurées, utilisé dans les messages MQTT.
- MQTT (Message Queuing Telemetry Transport)** Protocole de messagerie léger basé sur un modèle publication/abonnement, adapté à l'IoT et aux réseaux à faible bande passante.
- Power Supply** Alimentation électrique haute puissance convertissant et distribuant l'énergie vers les rails de charge.
- Qt** Framework multiplateforme de développement d'interfaces graphiques en C++, largement utilisé dans les systèmes embarqués.
- Recharge dynamique** Mode de recharge où le véhicule reçoit de l'énergie en roulant sur un rail électrifié.
- Recharge statique** Mode de recharge où le véhicule est à l'arrêt sur un rail électrifié ou un bumper.
- Systemd** Gestionnaire de système et de services pour Linux, responsable de l'initialisation du système et de la gestion des processus.
- Topic MQTT** Chaîne de caractères hiérarchique identifiant un canal de communication dans le protocole MQTT (ex : EVIAS/ei/1/status).
- TSlib (Touchscreen Library)** Bibliothèque Linux pour la calibration et la gestion des écrans tactiles.
- Wildcard MQTT** Caractère spécial (+ ou #) permettant de s'abonner à plusieurs topics simultanément selon un motif.

Annexe A

Annexes

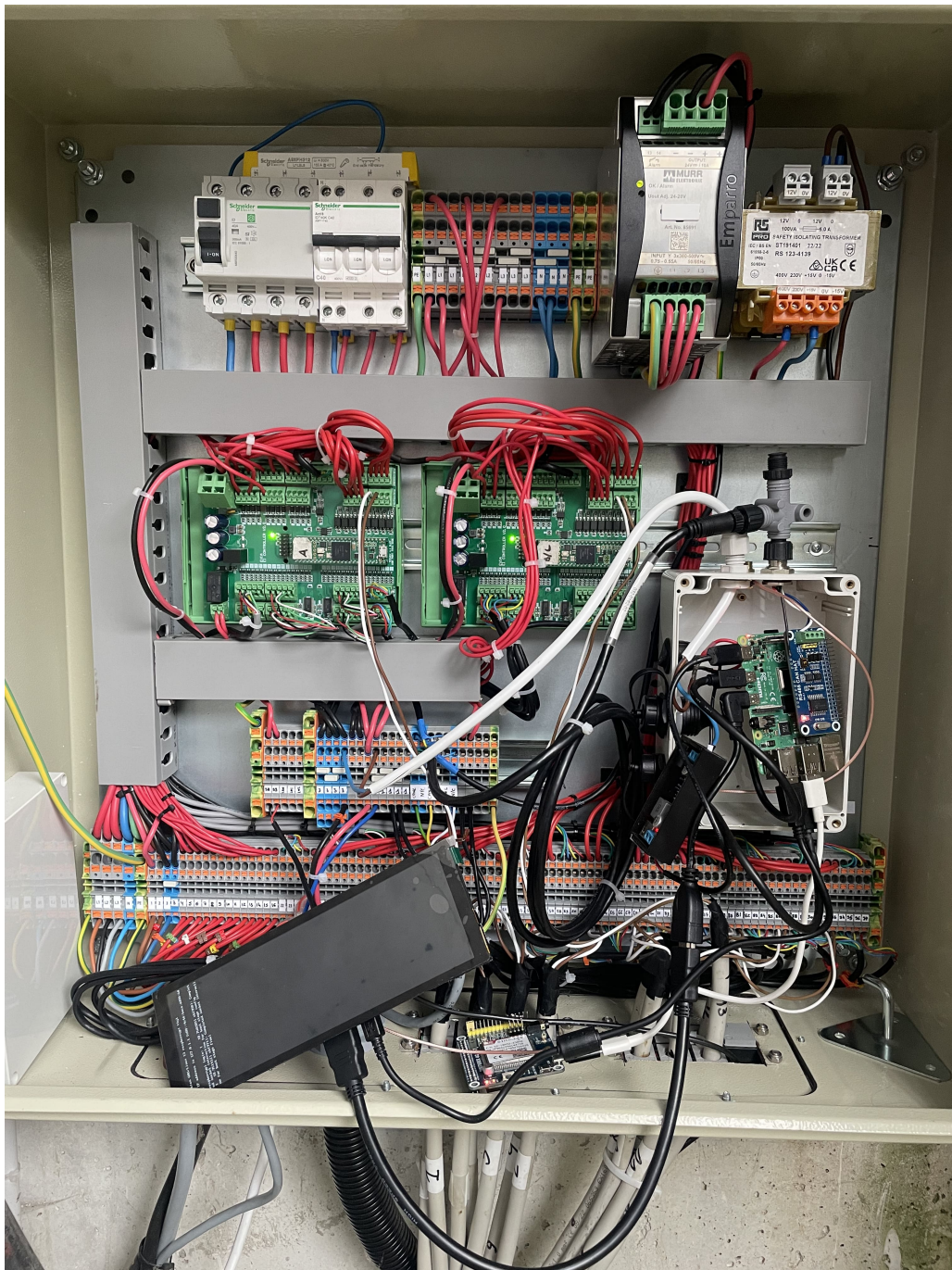


FIGURE A.1 – Écran tactile dans l'infrastructure (Simulation)

A.1 Schémas techniques

A.1.1 Diagramme UML complet

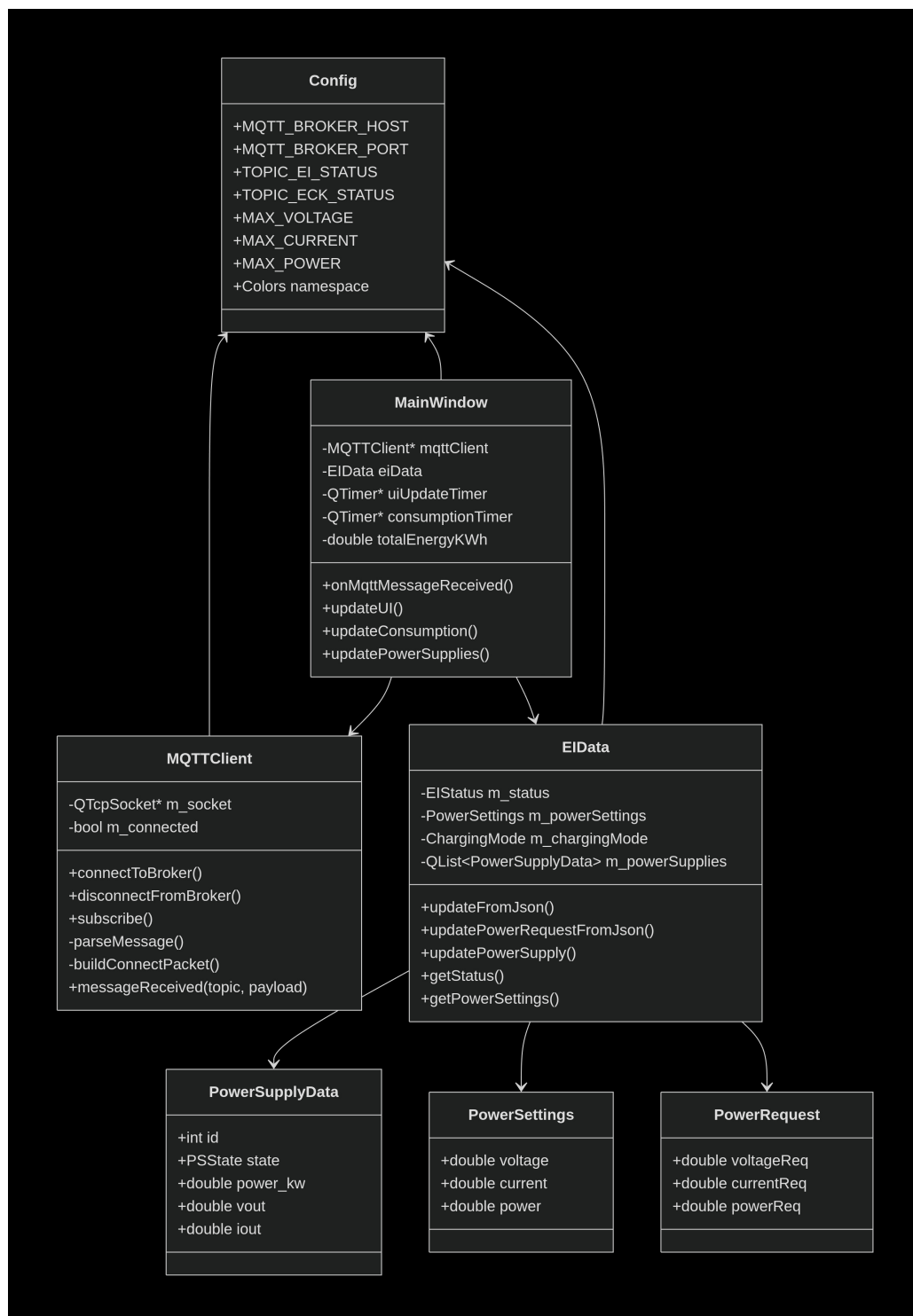


FIGURE A.2 – Diagramme UML

A.1.2 Flux complet des données MQTT

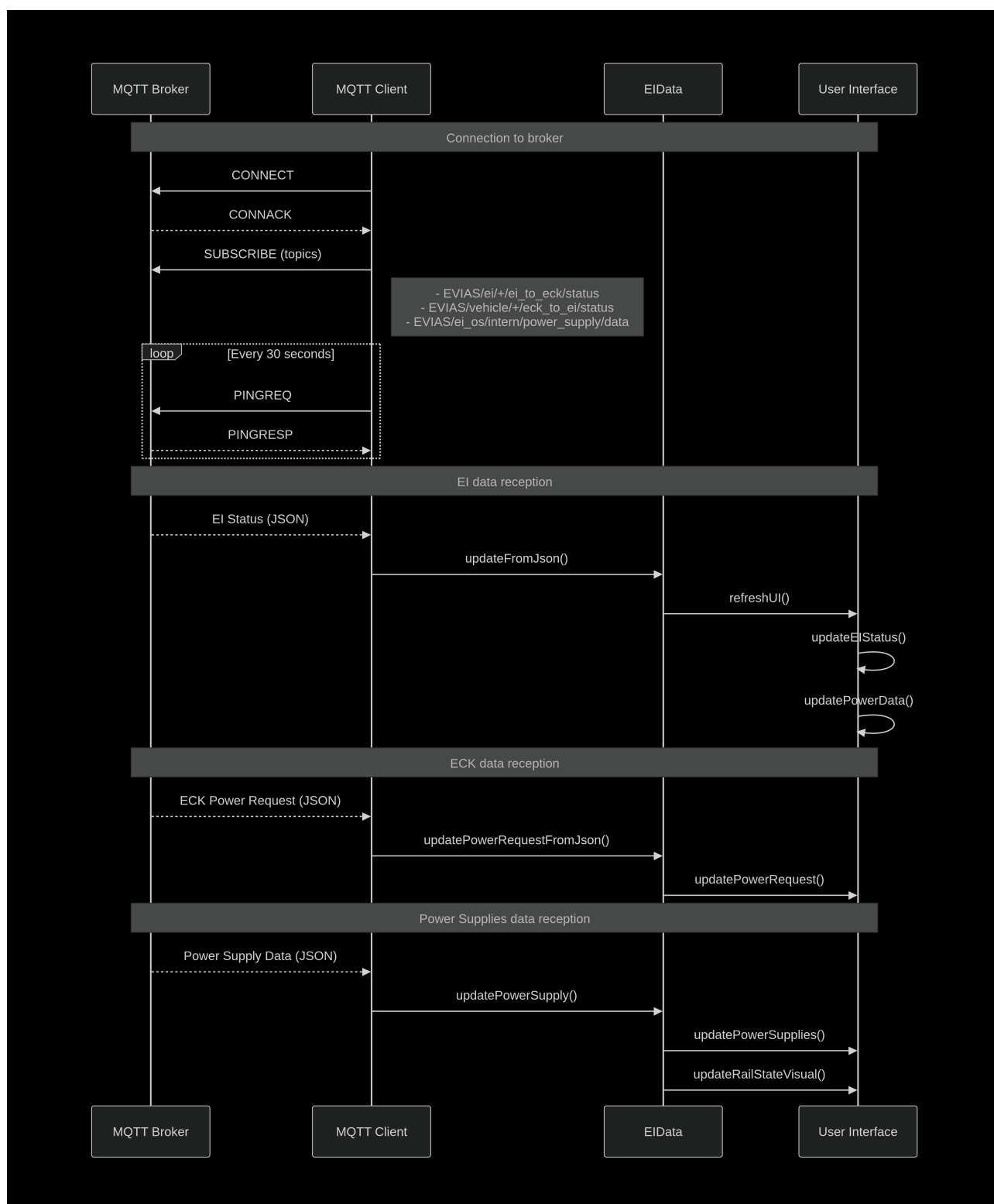


FIGURE A.3 – Architecture détaillée des flux de communication MQTT

A.1.3 Cycle de vie des données MQTT

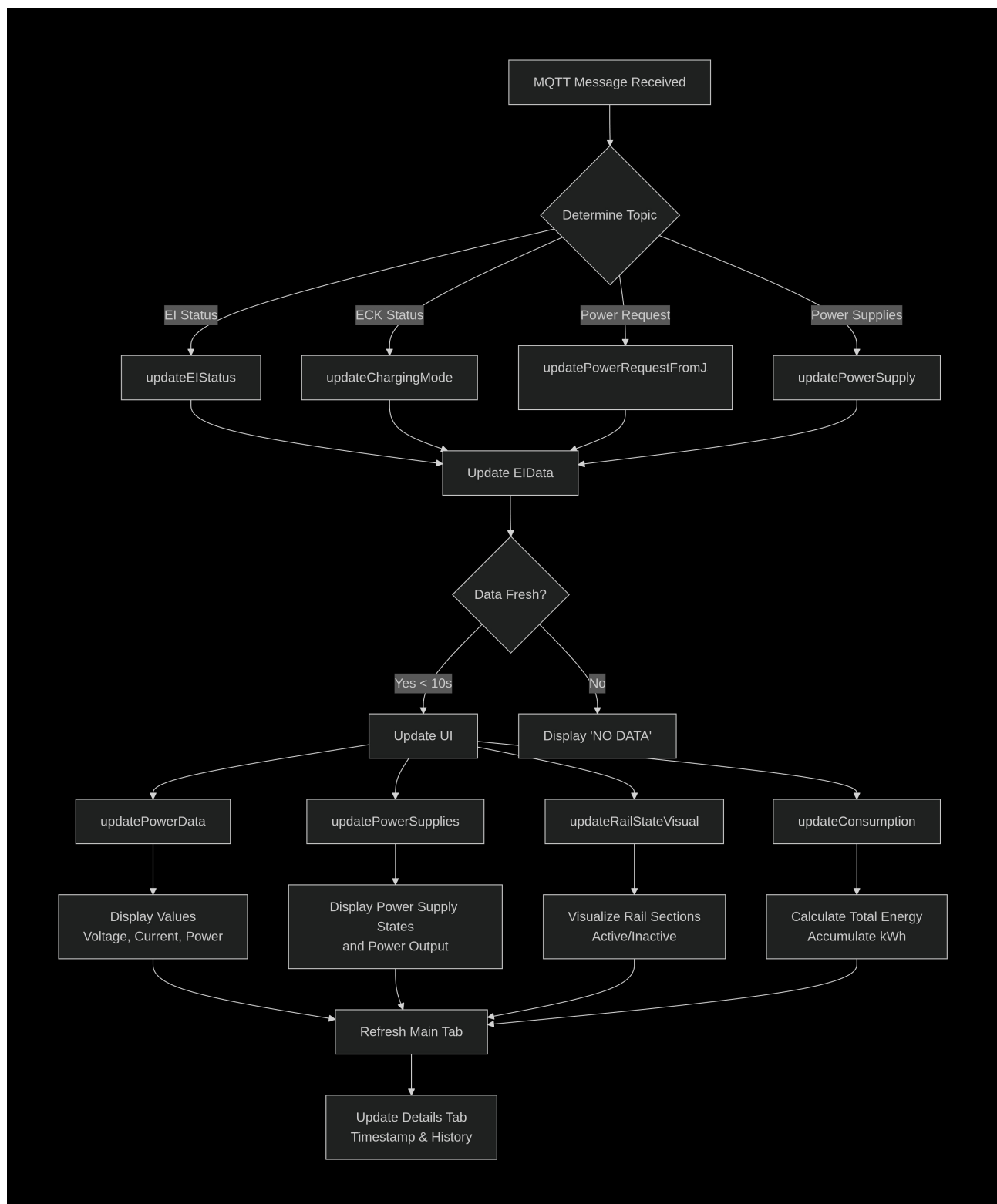


FIGURE A.4 – Cycle de vie des données MQTT

A.2 Configuration Systemd

A.2.1 Service splash screen

Listing A.1 – splashscreen.service

```
1 [Unit]
2 Description=Splash Screen EVIAS
3 DefaultDependencies=no
4 After=local-fs.target
5
6 [Service]
7 ExecStartPre=/usr/bin/chvt 1
8 ExecStart=/usr/bin/fbi -d /dev/fb0 --noverbose -a /home/evias/splash.
   png
9 StandardInput=tty
10 StandardOutput=tty
11 TTYPath=/dev/tty1
12 Type=oneshot
13
14 [Install]
15 WantedBy=sysinit.target
```

A.2.2 Service application HMI

Listing A.2 – evias_ei_hmi.service

```
1 [Unit]
2 Description=EVIAS HMI Application
3 Conflicts=splashscreen.service
4 After=local-fs.target
5
6 [Service]
7 User=root
8 WorkingDirectory=/home/evias/EVIAS_EI_HMI/build
9
10 Environment=QT_QPA_PLATFORM=eglfs
11 Environment=QT_QPA_EGLFS_ROTATION=-90
12 Environment=QT_QPA_GENERIC_PLUGINS=tslib
13
14 ExecStart=/home/evias/EVIAS_EI_HMI/build/EVIAS_EI_HMI
15
16 StandardInput=tty
17 StandardOutput=tty
18 TTYPath=/dev/tty1
19 Type=simple
20
21 Restart=always
22 RestartSec=2
23
24 [Install]
25 WantedBy=multi-user.target
```

English Summary

Development of a Human-Machine Interface for Electric Charging Infrastructure Monitoring

This eight-week internship at EVIAS, an innovative company developing dynamic and static electric vehicle charging solutions, focused on creating a centralized monitoring interface for the charging infrastructure.

Context and Objectives

EVIAS develops electric road technology that enables vehicles to charge while driving (dynamic charging) or while stationary (static charging) through a conductive rail embedded in the pavement. Before this project, the infrastructure lacked a centralized monitoring system, making it difficult for operators and engineers to have a real-time overview of the system's status.

The objective was to develop a Human-Machine Interface (HMI) displaying critical infrastructure information : operational status, power data, charging mode, energy consumption, power supply states, and rail section status.

Technical Approach

The interface was developed in Qt/C++ and deployed on a Raspberry Pi 4 with a 7.9" Waveshare touchscreen. The system architecture is based on MQTT protocol for real-time communication between the infrastructure control system (written in Python), the MQTT broker, and the Qt interface.

Key technical achievements include :

- Development of a custom MQTT client using raw TCP sockets (QtMqtt library unavailable)
- Implementation of real-time JSON message parsing
- Three-tab interface : MAIN (operator view), DETAILS (history), DEBUG (technical data)
- Display of 5 power supplies with state and instantaneous power
- Visualization of 14 rail sections with active section highlighting
- Energy consumption calculation (kWh) with reset capability
- Multi-vehicle support using MQTT wildcards

Integration and Deployment

The system integration involved several technical challenges :

- Waveshare screen configuration : touchscreen calibration (TSlib), Qt EGLFS setup
- Boot splash screen implementation using FBI and Systemd

- Automatic application launch via Systemd service
- Network management : 4G/WiFi coexistence with NetworkManager

Results

The project was successfully completed before the deadline, allowing the addition of features requested by engineers during weekly meetings. The interface is now operational and deployed on the Troyes test infrastructure.

This internship enabled me to develop expertise in embedded systems, MQTT protocol, Linux system configuration, and Qt framework.

The work documentation includes complete MQTT topic mapping and comprehensive source code comments, facilitating future maintenance and evolution of the system.